

1. ไม่ satisfy the evaluation criteria เนื่องจาก

- **Correctness** : code นี้ไม่มีการกันรั้ว Mutual Exclusion
ยกตัวอย่าง เช่น หากมีการ interrupt Thread 0 ก่อนจะ set flag[0] = 1 และหลังจากการตรวจสอบ flag[1] แล้วทำงานต่อไปใน Thread 1 เลข จะสังเกตได้ว่าทั้ง 2 Thread จะสามารถเข้าทำงานใน critical section ได้ในเวลาเดียวกัน ซึ่งยังทำให้เกิด Race Condition ได้
- **Performance** : code นี้ยังมีการใช้ spin-wait เพื่อรอการทำงานซึ่งอาจทำให้ต้องใช้เวลานานและมี CPU utilization ต่ำ เนื่องจากต้องมาวน loop spin รอเข้าไปทำงานใน critical section แทนที่จะใช้ CPU ในการทำงานอื่น โดย performance จะขึ้นอยู่กับการทำงานบน single processor ถ้าหากเป็น multi-core หรือ multiprocessor อาจมี performance ได้ดีกว่านี้
- **Fairness** : code นี้ไม่สามารถกันรั้วได้ว่า thread ที่รออยู่จะได้เข้าทำงานใน critical section อาจจะต้อง spin รอไปเรื่อย ๆ ซึ่งนำไปสู่ปัญหาน้ำ starvation

1. (15 Min) The following code attempts to implement a locking mechanism for two threads using two shared flags declared as:

```
int flag[2];
```

which are both initialized to 0. Both threads continuously execute a loop which contains a critical section of code.

Thread 0

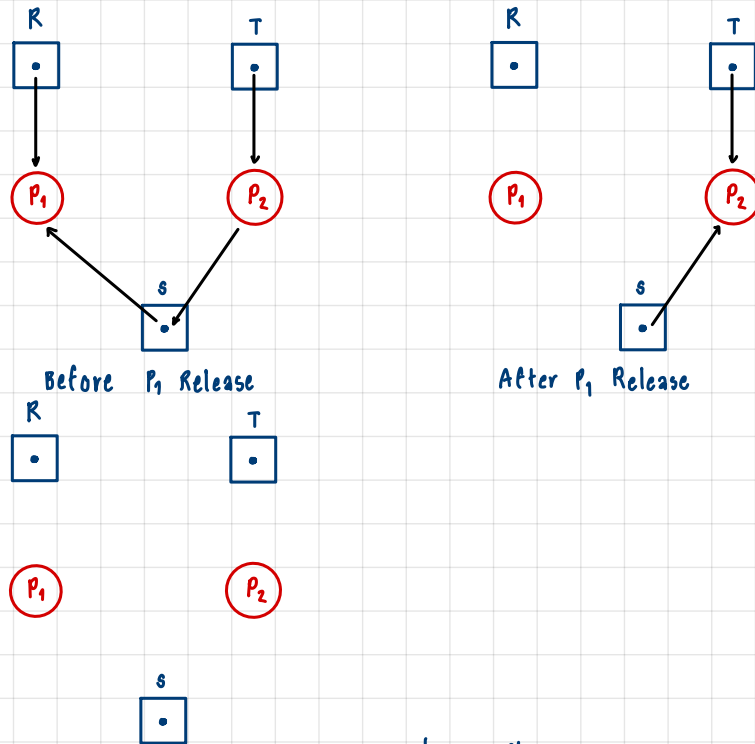
```
while (1) {  
    while (flag[1] == 1); // spin  
    flag[0] = 1;  
    // start critical section  
    ... ;  
    // end critical section  
    flag[0] = 0;  
    ... ;  
};
```

Thread 1

```
while (1) {  
    while (flag[0] == 1); // spin  
    flag[1] = 1;  
    // start critical section  
    ... ;  
    // end critical section  
    flag[1] = 0;  
    ... ;  
};
```

Does this code satisfy the evaluation criteria? If not, which of the requirements are not met? Explain your answer.

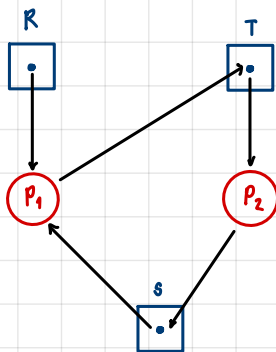
2. case 1



Final Resource Allocation → เนื่องจากไม่เกิด deadlock ทุก process จึงสามารถทำงานจนเสร็จ

∴ ไม่เกิด deadlock เนื่องจากไม่เกิด cycle หรือ circular wait ใน graph

case 2



∴ เกิด deadlock เนื่องจากเกิด 4 condition โดยที่สามารถเห็นได้จากใน graph คือ

- Hold and wait
- P₁ Hold R, S and wait for T
- P₂ Hold T and wait for S
- Circular wait (มี cycle ใน graph)

2. (15 Min) There are three resource types, R, S, and T, each with a single instance and there are 2 processes P1 and P2. Use Resource-Allocation Graphs to model the following situations. In each case, **determine** if there is a deadlock. You need **only show** the final resource allocation graph for each case in your answer.

Case 1

P1 requests R
P2 requests T
P1 requests S
P2 requests S
P1 releases R
P1 releases S

Case 2

P1 requests R
P2 requests T
P1 requests S
P2 requests S
P1 requests T

Part2-1

1. What is the benefit of thread over process when context switching occurs?
2. Explain how can a race condition effect the result of concurrency execution?
3. Give an example of code to create a lock from test_and_set()
4. How can you eliminate the time wasted in spinning loop in lock?

1. วิธีการ พังทึบหน่วยการมาทำ ทำให้ การสลับระหว่าง thread เกิดขึ้น ได้ทันที โดยไม่ต้องรอถึงระดับการ และ การสลับสว่สไป หรืออื่น

2. ส่วนผลที่เกิดขึ้น ความไม่สอดคล้อง กันของ ข้อมูล,

หรืออุปสรรคที่ยุ่งยาก, พฤติกรรมที่คาดเดาไม่ได้ และ มีการเกิด deadlock

(เนื่องจาก หลาย thread พยายาม แก้ไขตัวแปรที่เร้วพร้อมกัน)

เช่น thread นี้รออ่านค่า จากตัวแปร และใช้ผลลัพท์
ในขณะเดียวกัน thread อื่นก็ อาจเขียนผลลัพท์ขึ้นมาใหม่ ทำให้ เกิดการทับซ้อน
ส่วนผลนี้ การอุปสรรคที่ยุ่งยาก

```
3. void lock( lock_t * lock ) {  
    while (test_and_set (&lock -> flag, 1) == 1) ;  
}  
void unlock (lock_t * lock) {  
    lock->flag = 0 ;  
}
```

4. ใช้ sleep() เพื่อไม่ให้ไฟรอ เมื่อล๊อค ถูกป้อนเข้ามาทำงาน วิธีนี้ใช้ลด spin ได้

1. ทรัพยากร share resource เช่น Data โดยจะแยกแค่ stack, register และ PC ทำให้อุปสรรคทรัพยากรมากกว่า และ เว้นกว่า

2. ทำให้อัปเดตทรัพยากรเฉพาะค่าที่ผิดพลาด เนื่องจากทรัพยากร share Variable เช่น

counter = 0

Thread 1

```
load counter (0)
add 1 (1)
store counter (1)
```

Thread 2

```
load counter (0)
add 1 (1)
store counter (1)
```

expected = 2

```
3. void lock (lock_t * lock) {
    while (TestAndSet (&lock->flag, 1) == 1);
}

void unlock (lock_t * lock) {
    lock->flag = 0;
}
```

4. ใช้ sleep() เพื่อเฝ้าเข้าโปร เมื่อคิวว่างก็จะปลุกขึ้นมาทำงาน
วิธีนี้จะช่วยลดการใช้ spin ได้

How can we prevent deadlock in dining philosopher problem? Explain in detail.

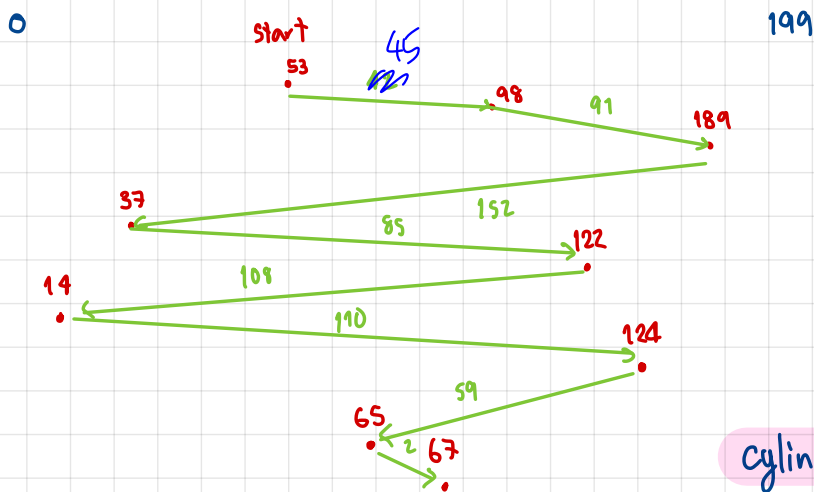
ให้ philosopher แย็บส้อมแบบ increasing order อยาเว้น philosopher คนสุดท้าย เพื่อทำลาย circular wait และเมื่อจะกินให้เช็คว่ามีส้อมเพียงพอต่อการกินหรือไม่ เพื่อไม่ให้เกิด Hold and Wait และเมื่อทุกคนกินเสร็จให้วางส้อมคืนให้คนอื่นสามารถกินต่อได้ ด้วยวิธีนี้ จะทำให้ไม่เกิด dead lock และทุกคนสามารถทำงานได้

Part2-3

If a disk has cylinder 0-199, and a head starts at position 53. The following positions are in reading queue {98,189,37,122,14,124,65,67}. Illustrate the seek path and calculate the total head movement for the following algorithms:

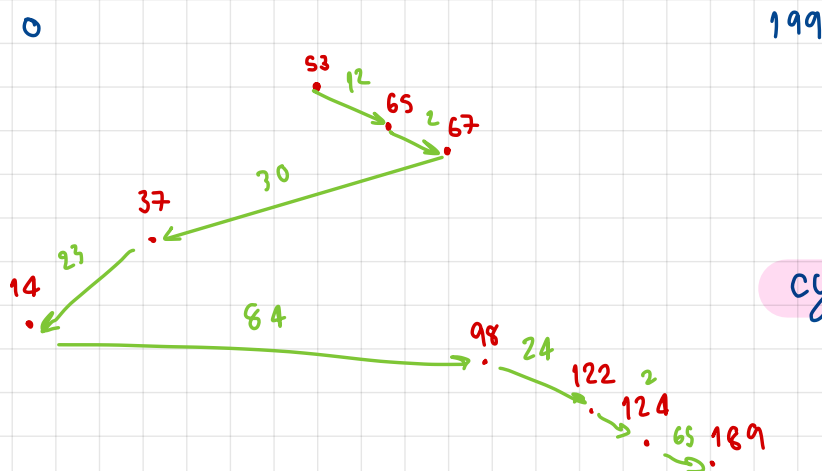
1. FCFS
2. SSTF
3. C-SCAN (left to right)
4. C-LOOK (left to right)

1. FCFS {98, 189, 37, 122, 14, 124, 65, 67}



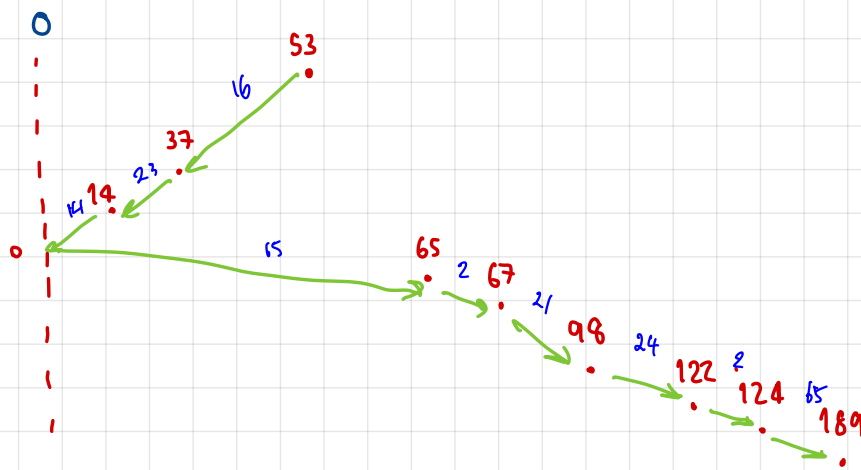
cylinders = 652.

2. SSTF {65, 67, 37, 14, 98, 122, 124, 189}



cylinders = 242

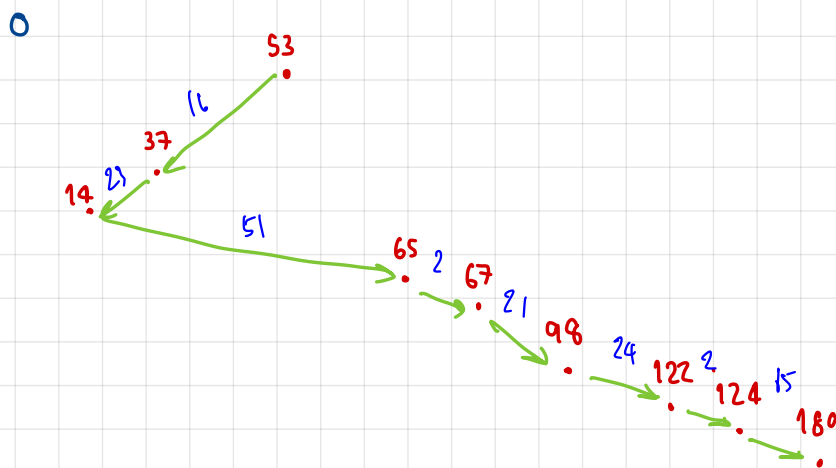
3. C-SCAN { 37, 14, 65, 67, 98, 122, 124, 189 }



199

cylinders = 242

4. C-LOOK { 37, 14, 65, 67, 98, 122, 124, 189 }



199

cylinders = 214

QUIZ (before final) on 24th November, 2020. (15:30 – 14:30)

1. Answer the following questions with precise and concise

a. Explain the difference between *Thread* and *Process*.

Process คือ สิ่งที่มี package ของ & component คือ Program, Control state
Data state และ Process status

ส่วน Thread คือ ส่วนที่ถูกแบ่งย่อยออกมาจาก Process ซึ่งภายใน 1 process
สามารถมีได้มากกว่า 1 Thread

b. Explain how *Test-and-Set* instruction can guarantee *Mutual Exclusion*.

ในกระบวนการของ Test-and-Set จะมี instruction ที่ return ค่า memory location
เก่าและ update ค่าใหม่เป็น 1 พร้อม ๆ กัน ซึ่งข้อจำกัดคือ 2 instruction ที่ทำงาน
เป็น Atomic Instruction ทำให้มันได้เกิด Mutual Exclusion

c. Explain the difference between *Deadlock* and *Starvation*.

Deadlock และ Starvation เป็นเงื่อนไขเหมือนกัน แต่ต่างกันที่

Deadlock เป็นเงื่อนไขที่ไม่มี Process ใดสามารถทำงานได้เนื่องจาก มีการขอ resource
ที่ถูกใช้งานโดย Process อื่นอยู่

ส่วน Starvation คือการที่ process ที่มี high priority มีการขอ resource
Process ที่มี low priority จนทำให้บาง process ไม่สามารถทำงานได้เลย

2. The following code attempts to implement a locking mechanism for two threads using two shared flags declared as:

```
int flag[2];
```

which are both initialized to 0. Both threads continuously execute a loop which contains a critical section of code.

Thread 0

Thread 1

```
while (1) {                                while (1) {
    while (flag[1] == 1); // spin
    while (flag[0] == 1); // spin
    flag[0] = 1;                            flag[1]
    = 1;
    // start critical section
    // start critical section

    ... ;                                ... ;
    // end critical section                //
    end critical section
    flag[0] = 0;                            flag[1] =
    0;
    ... ;                                ... ;
};                                         };
```

Does this code satisfy the evaluation criteria? If not, which of the requirements are not met? Explain your answer.

ถาม Code ในไคร่รีไม่ satisfy ตาม evaluation criteria ในเรื่องอะไรบ้าง

Correctness ในการทำงานอาจจะทำให้เกิดในหลายๆตัวภาพใช่ ตัวนี้ Code นี้ไม่เกิด

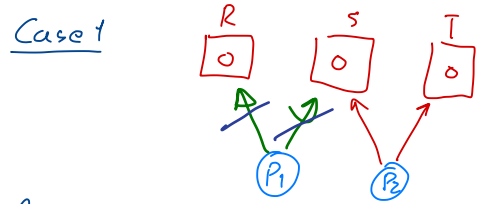
Mutual Exclusion

Thread 1	flag[0], flag[1] = 0	Thread 2
while (1) { while (flag[1] == 1); // spin		while (1) { while (flag[0] == 1); // spin flag[1] = 1;
flag[0] = 1; // start critical section ... ; // end critical section flag[0] = 0; ... ;		
};		// start critical section ... ; // end critical section flag[1] = 0; ... ; };

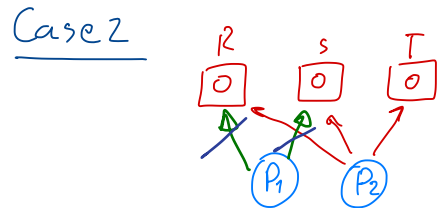
Performance ในด้าน performance เพื่อลดการใช้ spin-wait ทำให้
มี performance ที่ต่ำ เพื่อลดการใช้ CPU ในการ spin

3. There are three resource types, R, S, and T, each with a single instance and there are 2 processes P1 and P2. Use resource-allocation graphs to model the following situations. In each case, determine if there is a deadlock. You need only show the final resource allocation graph for each case in your answer.

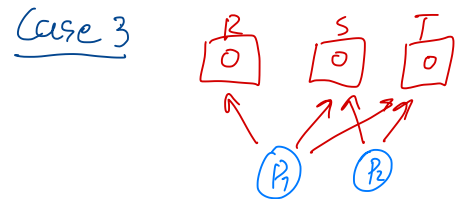
<u>Case 1</u>	<u>Case 2</u>	<u>Case 3</u>
P1 requests R	P1 requests R	P1 requests R
P2 requests T	P2 requests T	P2 requests T
P1 requests S	P1 requests S	P1 requests S
P2 requests S	P2 requests S	P2 requests S
P1 releases R	P2 requests R	P1 requests T
<u>P1 releases S</u>	<u>P1 releases R</u>	
	<u>P1 releases S</u>	



ใน Case 1 ไม่มี Deadlock



ใน case 2 ไม่มี Deadlock



ใน case 3 เป็น Deadlock เพราะว่า

P1 ขอ resource T จาก P2 แล้ว

P2 ขอ resource S จาก P1 จึง

ทำให้เกิด Deadlock

13 00 2560



King Mongkut's University of Technology Thonburi

**Final Exam
CPE334 Operating Systems**

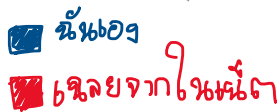
Tuesday May 9th: 13.00-16.00

Instructions

- This is a 12-page exam, please check that you have all the pages before start working.
 - The answers must be written in the space provided in this exam paper. Please read the instruction carefully.
 - A calculator and a paper dictionary are allowed.
 - A single A4-size note make be taken into the examination room (Write on both sides).
-

This examination is designed by Assoc.Prof.Dr. Tiranee Achalakul

Name _____ ID _____



Multiple choices (10 points)

1. Concurrency plays a major part in which of the following specific contexts:
 - a. Multiple applications
 - b. Structured applications
 - c. O/S structure
 - ☒ d. All of the above
2. Examples of solutions to the concurrency problem that do not involve busy waiting are the following:
 - a. Semaphores and monitors
 - b. Message passing and caching
 - c. Producers and consumers
 - ☒ d. None of the above
3. In order to implement mutual exclusion on a critical resource for competing processes, only one program at a time should be allowed:
 - ☒ a. in the critical section of the program
 - b. To perform message passing
 - c. To Exhibit cooperation
 - d. None of the above
4. The following requirement must be met by any facility or capability that is to provide support for mutual exclusion:
 - a. Only one process at a time can be allowed into a critical code section
 - b. A process remains in its critical code section for a finite time only
 - c. No assumptions can be made about relative process speeds
 - ☒ d. All of the above
5. In a uniprocessor system, mutual exclusion can be guaranteed by:
 - a. Overlapping processes
 - b. Interleaving processes
 - ☒ c. Disabling interrupts
 - d. All of the above
6. Which of the following disk seek algorithms has the most variability in response time?
 - a. First Come First Served
 - ☒ b. Shortest Seek Time first
 - c. SCAN
 - d. C-SCAN
7. RAID is a way to
 - a. Increase hard drive latency and performance.
 - b. Increase hard drive performance and decrease cost.
 - ☒ c. Increase hard drive reliability and performance.
 - d. Increase hard drive reliability and decrease cost.

8. Which of the following is not included in an inode in Linux?
 - a. File owner
 - b. File name**
 - c. File modification date
 - d. Pointer to the first data block.
9. Which of the following is not true for about the file system directory hierarchy in Linux?
 - a. Directories are stored in the same way as files
 - b. Directory entries are a fixed size
 - c. It can be represented as a directed graph**
 - d. A single file can have multiple names in the same directory.
10. Which of the following is not a benefit of using larger file blocks in a file system using indexed allocation?
 - a. Better disk space utilization
 - b. Faster access to a randomly-selected block of a large file
 - c. Higher average transfer rate for file data**
 - d. Less disk space consumed for metadata structures

Fill in the blank (10 points)

1. Many approaches to achieving mutual exclusion are software solutions that require the use of a technique called Semaphore.
2. When only one process is allowed in its critical code section at a time, then mutual exclusion is enforced.
3. The technique in which a process can do nothing until it gets permission to enter its critical section but continues to test the appropriate variable to gain entrance is called test-and-set.
4. The situation where Process 1 (P1) holds Resource 1 (R1), while P2 holds R2, and P1 needs R2 to complete and P2 needs R1 to complete is referred to as deadlock.
5. In order to protect shared variables (and other shared global resources) the system must control the Resource Allocation ?.
6. We use the concept of Files and Directories to abstract the file system.
7. The OS needs to create partitions and mount a file system before use
8. When the device has completed the operation, it will raise a hardware interrupt, causing the CPU to jump into the OS at a pre-determined ISR (Interrupt Service Routine)

มีใน
สไลด์

9. DMA (direct memory access)

is a hardware module that it helps the CPU with data movement between main memory and I/O devices

10. We need software in the OS that knows the details of how a particular device works. This type of software is called Device Driver.

Answer the following questions using only the space provided to write your answers. (70 points)

1. (2 points) Given that we can create user-level code to control access to critical sections, why is it important for an operating system to provide synchronization facilities such as semaphores in the kernel?

เพราะมันดีกว่า

Question is about offering sync services via the kernel than via user-level code.

To avoid busy waiting: the waiting thread can go to sleep
→ creates better cpu utilization; avoids priority inversion (if an issue)

NOT:

- The kernel needs to manage critical sections
- Software solutions might be buggy
- Semaphores are useful because ...

จะหลีกเลี่ยง spin-wait โดยให้ thread ของเรา sleep รอคนอื่น

2. (2 points) What is a race condition? Give an example.

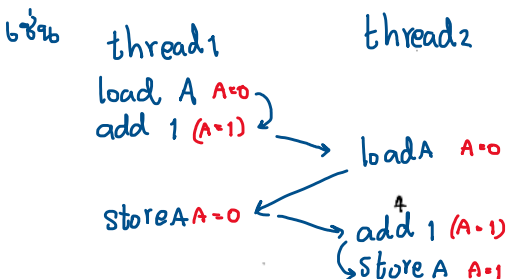
google is my friend

16. What is a race condition? Give an example.

When two (or even more) processes try to concurrently access a shared resource, leading to odd behaviour, deadlocks or mistakenly overwriting memory addresses.

Typical example from lectures: Two processes simultaneously updating a counter (lol, Assignment 1 math.c). Say Process A reads the variable but process B steps in and reads and writes to it. Control is passed back to process A, who just writes to it. You've just written to it twice. Ouch.

เมื่อ thread 2 thread ของเราไปแก้ไขค่าของ shared data ของเราแล้ว



Condition variable is an explicit queue that thread can put themselves on when some state of execution (i.e. some condition) is not as desired (by waiting on the condition; some other thread, when it changes said state can then wake one (or more) those waiting threads and thus allow them to continue (by signaling on the condition)

3. (6 points) Compare the two concepts of 'conditional variable' and 'semaphore' by given definition for both terms and explicitly state the differences between the two.

conditional variable

↳ ใช้ synchronization

Semaphore

↳ ใช้ทั้ง mutual exclusion & synchronization

Semaphore is an object with an integer value that we can manipulate with two routines

4. (3 points) Give an example (with explanation) of a practical use for a semaphore concept. In your given example, a simple mutex should not be sufficient and a semaphore is then required.

producer/consumer ใช้ mutex ไม่ใช้ semaphore = 2 thread ไม่สื่อสารกัน

Problem คือ check condition ri buffer
ใช้ shared variable
ใช้ semaphore

```
void *producer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&mutex);
        sem_wait(&empty);
        put(i);
        sem_post(&full);
        sem_post(&mutex);
    }
}

void *consumer(void *arg) {
    int i;
    for (i = 0; i < loops; i++) {
        sem_wait(&mutex);
        sem_wait(&full);
        int tmp = get();
        sem_post(&empty);
        sem_post(&mutex);
        printf("%d\n", tmp);
    }
}
```

5. (2 points) What is a test-and-set instruction? How can it be used to implement mutual exclusion?

21. What is a test-and-set instruction? How can it be used to implement mutual exclusion? Consider using a fragment of pseudo-assembly language aid you explanation.

← ใช้ตาม

A test and set instruction is a method of using the lock-variable solution, but with hardware support (instructions are in ASM and called before entering any critical region). Prior to entering the critical region, it copies the lock variable to a register, sets it to 1 if it was 0 (free), acquire the lock and execute. If not, another process had it. By being called as ASM instructions, it is guaranteed to be atomic.

เป็น atomic instruction ที่ return ค่าเดิมและอัปเดตค่าใน memory

```
void lock(lock_t *lock) {
    while (TestAndSet(&lock->flag, 1) == 1);
}
```

เชื่อว่าถ้า flag เป็น 0 ก็เปลี่ยนค่า flag เป็น 1 แล้วหยุดรอ → ไม่ lock
ถ้า flag เป็น 1 จะติด spin-wait → lock

6. (6 points) The following Java code samples describe two Lock classes with two methods each: `acquire()` and `release()`. You can assume that the application calls `lock.acquire()` before entering a critical section and `lock.release()` after exiting the critical section. For the implementations that require a tid (i.e., thread id), you can assume that the tid of each thread is either 0 or 1.

<pre> class LockA { private int turn = 0; public void acquire(int tid) { while (turn == (1 - tid)); } public void release(int tid) { turn = (1 - tid); } } </pre>	<pre> class LockB { public void acquire() { disableInterrupts(); } public void release() { enableInterrupts(); } } </pre>
--	---

- Does the code guarantee mutual exclusion?
- Does the code guarantee progress?
- List all other limitations that exist for each implementation. Issues you might consider include (but are not limited to) the following: generality, efficiency, and fairness.

Lock A

a) mutual exclusion ได้

b) ไม่ progress

c) ไม่ spin - wait → บัสเวลา

Lock B

a) mutual exclusion ได้

b) มี progress

c)

- disableinterrupts (กับ enableinterruptsc) เป็น privilege operation
ใช้ตอนให้ user ใช้ hardware
- มาเปิด/ปิด interrupt ให้ efficiency $\Rightarrow$ ช้า
- ใช้ร่วมกับ multi-core

7. (5 points) Suppose there are two processes, A and B, which share semaphores R and S. At the starting time, $R = 0, S = 1$. A and B are entering the sections of code shown below:

Sem_P = wait
Sem_V = post

Process A:

Sem_P(R); **Can vvv**

Sem_P(S); **Lock**

Operation A.1;

Sem_V(S);

Operation A.2;

Sem_V(R);

Process B:

Operation B.1;

sem_V(R);

Operation B.2;

Sem_P(S);

Operation B.3;

Sem_V(S);

Each of the following statement, state whether it is true or false and give a brief reason

7.1 Operation A.1 cannot start until operation B.1 is complete.

true เพราะ ถ้า Process A เริ่มทำงานก่อนจะถูกระงับไว้ให้ B ทำงานก่อน

A: Sem-P(R) $\setminus R=0, A=100$

B: B1

ถ้า Process B เริ่มทำงานก่อน B ก็จะไม่เสร็จก่อนจนกว่า A จะเสร็จ

7.2 Operations A.1 and B.2 cannot execute simultaneously.

A: Sem-P(R) $\rightarrow R=0$

B: B1

B: Sem-V(R) $\rightarrow R=1$

A: Sem-P(R) $\rightarrow R=0$

A: A1

B: B2

false มีโอกาสที่จะทำงานพร้อมกันได้

เพราะ Sem-V() ทำให้อุปกรณ์ A มาทำงาน

ใช้ไม่ได้ใน B ทีเดียว

↓

7.3 Operations A.1 and B.3 cannot execute simultaneously

B: Sem-P(R) $\rightarrow R=0$

A: Sem-V(R) $\rightarrow R=1$

A: A1

true เพราะ Sem-P() จะทำให้อุปกรณ์ B มาทำงาน

จนกว่า A.1 จะทำงานเสร็จแล้วจึงเรียก Sem-V()

ดังนั้นจึงสามารถทำงานพร้อมกันได้

7.4 Operations A.2 and B.3 cannot execute simultaneously

false เพราะ หลังจาก A ใช้ Sem-V() แล้วก็ทำ A.2 ต่อได้

เพราะ Sem-V() จะไปปลด B ให้มาทำ B.3 ต่อด้วย

7.5 The system may enter the deadlock state.

false

8. (2 points) Explain what is meant by *starvation* in the context of a solution to the mutual exclusion problem.

คือ thread ที่เข้าคิว lock แต่ไม่ unlock ทำให้ thread ใ้ไม่ได้ทำงาน

9. (2 points) Briefly describe the concept of producer/consumer (bounded buffer) and give one example of real world application that fits the concept.

producer ៖ check ថា buffer តែងឆ្លើយ → ទុកទីតាំង → put
consumer ៖ check ថា buffer តែងឆ្លើយ → ទុកទីតាំង → get

In multithread web server, a producer put HTTP request into a work queue; consumer thread take requests out of this queue and process them

10. (2 points) Give a definition of a term "virtual machine".

mid term ១:

11. (3 points) compare and contrast the concepts of "full virtualization" and "para-virtualization".

១២១២២២២

12. (3 points) What are the benefits of virtualization, and what is one situation where you would want to use a VM instead of a physical machine?

mid term ២

13. (2 points) describe the key concept of JAVA virtual machine and its main benefit.

วิธี

14. (3 points) Assuming the operating system detects the system is deadlocked, what can the operating system do to recover from deadlock?

31. Assuming the operating system detects the system is deadlocked, what can the operating system do to recover from deadlock?

ตายแน่

The operating system can recover from deadlock by:

- **Killing the deadlocked process** (crude, and requires that it will be restarted)
- **Rolling Back** (going back a few instructions until it can enter a non-deadlocked state, and retry)
- **Pre-emption** (forcing the handing over of a resource to another process, at the expense of the one it's currently deadlocked on)

หยุดทุก Process ที่เกี่ยวกับ deadlock หรือ หยุด 1 Process ที่จะทำให้หยุด dead lock

15. (7 points) Recall the various deadlock detection and prevention algorithms we've discussed, and consider the following snapshot of a system with five processes (P1, P2, P3, P4, P5) and four resources (R1, R2, R3, R4). There are no current outstanding queued unsatisfied requests.

Currently Available Resources

R1	R2	R3	R4
2	1	2	0

Process	Current Allocation				Max Need				Still Needs			
	R1	R2	R3	R4	R1	R2	R3	R4	R1	R2	R3	R4
P1	0	0	1	2	0	0	3	2	0	0	2	0
P2	2	0	0	0	2	7	5	0	0	7	5	0
P3	0	0	3	4	6	6	5	6	6	6	2	2
P4	2	3	5	4	4	3	5	6	2	0	0	2
P5	0	3	3	2	0	6	5	2	0	3	2	0

4 6 12 12 16 23 14

not deadlock

P₁ P₄ P₅ P₂ P₃

- a) Is this system currently deadlocked, or can any process become deadlocked? Why or why not? If not deadlocked, give an execution order.

P₂, P₃, P₄, P₅ มีโอกาสเกิด deadlock

- b) If a request from a process P₁ arrives for (0, 4, 2, 0), can the request be immediately granted? Why or why not? If yes, show an execution order.

ไม่ได้อะไร เพราะ R₂ มี instance เหลือไม่ถึง 4

16. (3 points) Give a scenario where choosing a large filesystem block size might be a benefit; give an example where it might be of disadvantage.

ข้อนี้เลย

36. Give a scenario where choosing a large filesystem block size might be a benefit; give an example where it might be a hinderance.

Large file system block sizes are a help (where performance is concerned) when the files in question are very large, and contiguous reading/writing is prevalent. For example, multimedia files. A large file system block size, unfortunately, wastes a lot of space if your file system largely consists of small sized files. Random access to small segments of data require loading entire blocks of data, even though you only need a small amount of it.

ดี → access ที่เร็ว ได้ข้อมูลเยอะ

ไม่ดี → เปลือง (internal fragmentation)

17. (3 points) What is the reference count field in the inode? You should consider its relationship to directory entries in your answer.

count ว่า มีกี่ directory ที่ reference ถึง file นี้ได้

ข้อนี้

46. What is the reference count field in the inode? You should consider its relationship to directory entries in your answer.

The reference count field is a counter of how many times the inode is "referenced" by name. Adding a directory entry increments this counter. When the count falls to zero, (i.e. there are no longer any directory entries to that file), its inode and all corresponding disk blocks can be safely deallocated.

18. (7 points) In class, we discussed three different techniques for organizing the data blocks for each file in a file system, namely contiguous allocation, linked allocation, and indexed allocation. Briefly describe each approach, identifying the strengths and weaknesses of each

1. Contiguous

1 file มี block ติดกัน

ดี → performance สูง

→ access ได้ทั้ง sequence

เลว = random

ไม่ดี → มี external fragment

→ วนวน file ยาก

2. linked

1 block จะมี pointer ชี้ไป

ที่ block ต่อไปใน file

ดี → ไม่มี external fragment

ไม่ดี → access แบบ random ไม่ได้

3. indexed

1 file มี 1 block ที่เก็บ pointer

ทั้งหมดใน file

ดี → access แบบ random

ไม่ดี → ใช้ pointer เยอะกว่า

19. (2 points) What file allocation strategy is most appropriate for random access files?

contiguous & indexed

แต่แบบ random

indexed →

39. What file allocation strategy is most appropriate for random access files?

An inode (indexed node) based file allocation strategy. It is a form of file allocation table that offers temporal locality – the inode only needs to be located in memory when its corresponding file is open. The inode is a data structure which lists all attributes and points to all the addresses of the disk block corresponding to that file.

20. (5 points) Compare I/O based on polling with interrupt-driven I/O. In what situation would you favor one technique over the other?

เดี๋ยวนี

70. Compare I/O based on *polling* with *interrupt-driven* I/O. In what situation would you favour one technique over the other?

Polling means that we are continually using CPU cycles to check whether any I/O is occurring or not. However, it means the CPU is busy-waiting on any other task so as to not miss any I/O. Interrupt-driven I/O allows the CPU to work on other tasks and handle requests on demand, however it requires a context switch into the interrupt handler to process it. We normally favour interrupt-driven I/O for any sort of human interface device, as it is much slower than the data processing it is doing, so a few hundred interrupts each second won't matter. We might favour polling for high bandwidth raw data transfer applications, as a million context switches tends to slow things down.

polling ใช้กับ device ที่ transfer data เร็ว

ส่วน interrupt ใช้กับ human interface device



King Mongkut's University of Technology Thonburi

Final Exam
CPE334 Operating Systems

Instructions

- This 11-page exam contains 6 questions for a total of 100 points.
- The answers must be written in this exam paper. Please read the instruction carefully.
- A calculator and a paper dictionary are allowed.
- A single A4-size note make be taken into the examination room (Write on both sides).

This examination is designed by Assoc.Prof.Dr. Tiranee Achalakul

Name _____ ID _____

Question 1	
Question 2	
Question 3	
Question 4	
Question 5	
Question 6	

Question 1: Multiple choices (10 Points)

Students get 1 point for each correct answer and -0.5 point for each wrong answer.

1. With DMA (Direct Memory Access):
 - a. The processor can read or write directly to a device.
 - b. The kernel can read or write directly to a process' memory without intermediate buffers.
 - c. A process can read or write to kernel memory without intermediate buffers.
 - ☒ d. The device can read or write directly to the system's memory.
2. To implement a user-level threads package, the operating system may need to provide:
 - ☒ a. Non-blocking system calls
 - b. Kernel threads
 - c. An exec mechanism
 - d. Direct memory access

User-level threads requires non-blocking systems call i.e., a multithreaded kernel. Otherwise, entire process will be blocked in the kernel, even if there are runnable threads left in the processes. For example, if one thread causes a page fault, the process blocks.
3. A compare-and-swap instruction allows you to:
 - a. Modify a memory location only if its contents match a given value.
 - ☒ b. Exchange the contents of two memory locations if their values are different.
 - c. Exchange the contents of two memory locations if a lock is not set.
 - d. Exchange the contents of two memory locations if a lock is set.
4. Starvation is the case when a thread:
 - a. Loops continuously until it runs out of memory.
 - ☒ b. is never scheduled to run.
 - c. Can never acquire a lock on a critical section.
 - d. Cannot create a child process or thread
5. A thread that is blocked on a semaphore is awakened when another thread:
 - a. Tries to decrement a semaphore's value below 0.
 - ☒ b. Tries to increment the semaphore.
 - c. Causes the semaphore's value to reach a specific number.
 - d. Tries to block on the same semaphore
6. Condition variables support these operations:
 - ☒ a. Wait / notify
 - b. Read-and-increment / wait-for-value
 - c. Increment / decrement-and-wait
 - d. Set-value / wait-for-value
7. Which statement is true:
 - ☒ a. Using mutual exclusion ensures that a system avoids deadlock.
 - b. The value of a semaphore can never be negative.
 - c. A Lock built with Test-and-Set can guarantee that a waiting thread will eventually enter the critical section.

d. Two phase locking use 'sleep' instead of 'spin' in its implementation

8. Which of the following is the allocation method of a disk space?

- a. Contiguous allocation
- b. Linked allocation
- c. Indexed allocation
- ☒ d. All of the above

9. At a particular time of computation, the value of a counting semaphore is 7. Then 20 'P' operations and 15 'V' operations were completed on this semaphore. The resulting value of the semaphore is:

- a. 42
- ☒ b. 2
- c. 7
- d. 12

$$\begin{array}{l} P \rightarrow \text{wait} - 20 \\ V \rightarrow \text{post} + 15 \end{array} \quad \left. \vphantom{\begin{array}{l} P \rightarrow \text{wait} - 20 \\ V \rightarrow \text{post} + 15 \end{array}} \right\} 7 - 5 = 2$$

10. The _____ present a uniform device-access interface to the I/O subsystem, much as system calls provide a standard interface between the application and the operating system.

- a. devices
- b. buses
- ☒ c. device drivers
- d. I/O systems

မေးခွန်း

Question 2: Short answer questions. Use only the space provided to write your answers. (10 points)

1. (2 points) Exceptions are events that stop normal execution, switch the execution mode into kernel mode, and begin execution at special locations within the kernel. Name two different types of exceptions and give an example of each type.

- divide by zero

- invalid memory access

2. (4 points) For each of the following thread state transitions, say whether the transition is legal. If so, explain how the transition occurs. If not, say why.

a) Change from thread state BLOCKED to thread state RUNNING

ไม่

}

มี semaphore

b) Change from thread state RUNNING to thread state BLOCKED

ใช่

มี semaphore
ให้ไปรอ

3. (2 points) Given that we can create user-level code to control access to critical sections, why is it important for an operating system to provide synchronization facilities such as semaphores in the kernel?

software จะเกิดปัญหา spin-wait

โดยต้องให้คำสั่งของ kernel มาช่วยทำให้ sleep รอแทน spin-wait

Question is about offering sync services via the kernel than via user-level code.

To avoid busy waiting: the waiting thread can go to sleep

→ creates better cpu utilization; avoids priority inversion (if an issue)

4. (2 points) Name at least three things that affect the speed of the disk access time.

transfer rate

rotation time

seek time

Question 3: (20 points)

3.1 (8 points) The methods 'Lock' (wait) and 'Unlock' (signal) must be atomic to ensure a correct implementation of mutual exclusion. Give an example scenario to show that if lock() is not atomic then mutual exclusion cannot be maintained. The scenario must be explained with both Pseudo-code and plain language (English or Thai)

implement lock ด้วยวิธี test and set

```
lock(flag) {  
    while(flag == 1)  
        flag = 1  
}
```

ตอนเริ่ม flag=1

สมมุติ มี process 1 เข้ามา lock ก็จะเห็นว่า flag=1 จึง

process 1: while(flag==1)

แต่ flag=0 อยู่จึงจะไปทำงานต่อ คือ set ค่า flag=1 แทน

ก่อนจะ set ค่าไป context switch ไป process 2

process 2 ก็จะเห็นค่า flag

process 2: while(flag==1)

ซึ่ง flag=0 อยู่จึงไปทำงานต่อ คือ set ค่า flag

process 2: flag=1

และมีการ context switch กลับไป process 1 ให้ทำงานต่อ

process 1: flag=1

ทำให้ไม่มีใคร spin-wait อยู่ใน lock และ 2 process ก็จะ
เข้าไปทำงานพร้อมๆ กัน

3.2 (8 points) Compare the two concepts of 'conditional variable' and 'semaphore' by given definition for both terms and explicitly state the differences between the two.

condition variable คือ การทำ synchronization
จัดการลำดับการทำงานของ thread

semaphore จะทำทั้ง synchronization และ lock ให้เป็น
mutual exclusion

3.3 (4 points) Give a scenario where choosing a large filesystem block size might be a benefit; give an example where it might be of disadvantage.

ข้อดี
↳ 1 block เก็บข้อมูลได้เยอะ
↳ access 1 block ได้ข้อมูลเยอะ
ข้อเสีย
↳ internal fragmentation

Question 4: (20 points)

Imagine a number of concurrent list operations, including Inserts and simple lookups.

1. Inserts change the state of the list, and thus only a single writer can acquire the lock to enter the critical section.
2. Lookups simply read the list; as long as we can guarantee that no insert is on-going, we can allow many lookups to proceed concurrently.

Write a pseudo-code to use the semaphore concept to implement a special type of lock to support this type of operation (known as *reader-writer lock*).

Hint: If some thread wants to update the list, it should call the new pair of synchronization operations:

- `rwlock_acquire_writelock()`, to acquire a write lock
- `Rwlock_release_writelock()`, to release it.

If some thread wants to read from the list, it should call:

- `rwlock_acquire_readlock()`, to acquire a read lock
- `Rwlock_release_readlock()`, to release it.

Now use semaphore to write a pseudo-code for these four functions.

```
sem_t empty, full, mutex;

sem_init(&empty, 0, 1); // if empty = 1 → list is empty
sem_init(&full, 0, 0); // if full = 0 → list is full
sem_init(&mutex, 0, 1); // if mutex = 1 → lock is free

void *rwlock_acquire_writelock() {
    sem_wait(&empty);
    sem_wait(&mutex);
}

void *Rwlock_release_writelock() {
    sem_post(&full);
    sem_post(&mutex);
}

void *rwlock_acquire_readlock() {
    sem_wait(&full);
    sem_wait(&mutex);
}

void *Rwlock_release_readlock() {
    sem_post(&empty);
    sem_post(&mutex);
}
```

Question 5: (15 points)

5.1 (5 points) Processor and I/O Controller can work together in two different ways. First, the processor uses generic I/O commands with Memory-mapped I/O. Second, processor sends special I/O commands directly to each I/O controller. Explain the mechanism of both methods and discuss their differences.

แบบ Memory-mapped I/O

↳ device register ถูก mapped ลง memory space
เพื่อใช้คำสั่ง store/load ปกติ

แบบ special I/O command

↳ ใช้คำสั่งพิเศษ read/write ไปที่ device register โดยตรง

5.2 (5 points) Compare I/O based on polling with interrupt-driven I/O. In what situation would you favor one technique over the other?

polling เหมาะกับ device ที่ transfer data เร็ว,
ไม่ต้องการเวลา interrupt

interrupt เหมาะกับ HCI (human computer interface)

5.3 (5 points) When a file is removed, the blocks it occupies are simply placed back onto the free list. Data can still be accessed with the correct tools, which may be problematic. How would you overcome such a problem? Describe your idea.

เวลา delete file คือจะลบชื่อfileกับinode ออกจาก directory
คือ จะมองไม่เห็น แต่จริงๆ file ยังอยู่
แก้ด้วยการทำ defragment หรือใช้ format เลย!

Question 6: (25 points)

6.1 (5 points) Explain briefly how to create your own system call step by step

1. สร้าง file code ของเรา
2. เพิ่ม file ลงใน kernel's makefile
3. เพิ่ม file ลงใน system call table
4. เพิ่ม system call ในไฟล์ syscall.h
5. compile kernel
6. update kernel

6.2 (5 points) Write a code to test a system call if the number of system calls is 323 and the function of a system call is show below

```
asm linkage long sys_mod (long a, long b)
{
    long answer;
    answer = a + b;
    return answer;
}
```

```
int main()
{
    long no = syscall(323);
    printf("sys_mod return %ld\n", no);
    return 0;
}
```

5. (15 points) Explain how you can achieve your task in assignment 4 (CPU, Memory, Threads, File system). The explanation must include source file name that you have edited.

9x10⁹ 2¹⁵
v v

เลขที่นั่งสอบ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

ข้อสอบปลายภาคการศึกษาที่ 2/2557

วันที่ 11 พฤษภาคม 2558

เวลา 09.00 – 12.00 น.

วิชา CPE 334 Operating System

น.ศ. วิศวกรรมคอมพิวเตอร์ปีที่ 3 หลักสูตรภาษาไทย

คำสั่ง

- ข้อสอบมี 2 ส่วน ส่วนละ 50 คะแนน รวมเป็น 100 คะแนน
 - ส่วนที่ 1 มี 3 ข้อใหญ่
 - ส่วนที่ 2 มี 3 ข้อใหญ่
- ให้ทำข้อสอบส่วนที่ 1 ลงในตัวข้อสอบที่เว้นช่องไว้ให้ ส่วนที่ 2 ทำลงในสมุดคำตอบ
- อนุญาตให้ใช้เครื่องคำนวณ รุ่นและยี่ห้อที่มีมหาวิทยาลัยกำหนดได้
- ไม่อนุญาตให้นำเอกสารใดๆ เข้าห้องสอบ
- เขียนชื่อและ รหัสประจำตัว ลงในกระดาษคำตอบทุกแผ่น (และแผ่นนี้)

(รศ.ดร.ธีรณี อจลากุล)

ผู้ออกข้อสอบส่วนที่ 1

(อ.ราชวิรัช สโรชวิกสิต)

ผู้ออกข้อสอบส่วนที่ 2

ข้อสอบนี้ได้ผ่านการประเมินจากภาควิชาวิศวกรรมคอมพิวเตอร์แล้ว

ชื่อ รหัสประจำตัว ภาควิชาชั้นปี

ชื่อ รหัสประจำตัว ภาควิชาชั้นปี

ส่วนที่ 1

1: (25 points) Short answers: write your answers within the provided space

1.1. What is the relationship between threads and processes?

process สามารถแบ่งเป็นหลาย thread ได้
 ทำให้สามารถทำ 1 process ได้เร็วขึ้นด้วยการทำ
 หลาย thread พร้อมกัน

1.2. What is a race condition? Give an example.

process หลาย process มาชนเขต shared variable หนึ่งอย่างกัน

1.3. What are the requirements of any solution to the critical sections problem? Why are the requirements needed? (hints: mention at least two requirements)

- mutual exclusion
↳ ต้องเข้ามาได้ทีละคน
- performance
↳ ไม่เปลือง CPU ตัดวงจร spin-wait
- fairness
↳ ได้ทำทุกอัน ไม่ starvation

1.4. What is a test-and-set instruction? How can it be used to implement mutual exclusion?

- เป็น atomic instruction
- จะ return ค่าของตัวแปร และ set ค่าใหม่ในตัวแปร
- ใช้ทำ loop spin-wait

while CTestAndSet(mutex → flag, 1) == 1

ชื่อ รหัสประจำตัว ภาควิชา/ชั้นปี

1.5. What is the producer-consumer problem? Give an example of its occurrence in operating systems.

producer จะทำหน้าที่ใส่ค่าใน buffer
consumer จะทำหน้าที่เอาค่าใน buffer ออกมา
ปัญหาคือ ต้องกันไม่ให้ producer ใส่ค่าถ้า buffer เต็ม
และ consumer เอาค่าออกมาจาก buffer เปล่า

1.6. What is a semaphores? Describe how to implement a lock using semaphores.

semaphore คือ ตัวแปรหนึ่ง ที่ใช้กับ คำสั่งพวกนี้

- sem-init → set ค่าเริ่มต้นของตัวแปร
- sem-wait → รอว่าตัวแปร < 0 มี ถ้า 0 ยอมที่จะให้ process ที่ call sleep
แต่ถ้า > 0 จะให้ทำงานได้ และ ลดค่าตัวแปร ไป 1 (lock)
- sem-post → ยกค่าตัวแปร ไป 1 แล้วปลุก process ที่นอนอยู่มาทำงาน (unlock)

1.7. What is deadlock? What is starvation? How do they differ from each other?

deadlock = ไม่มีใครได้ทำงานเลย

starvation = คนหนึ่งไม่ได้ทำงานสักที

ถ้าจะกันตรงที่ starvation ย่อมมีคนได้ทำงานแล้ว deadlock ไม่มีเลย

1.8. Give an example (with explanation) of a practical use for a semaphore concept. In your given example, a simple mutex is should not be sufficient and a semaphore is then required.

producer-consumer จะใช้ semaphore ในการ lock
และใช้ mutex ธรรมดาไม่ได้ เพราะ producer กับ consumer
ไม่ได้สื่อสารกัน

ชื่อ รหัสประจำตัว ภาควิชาชั้นปี

2: (10 points) From the code fragments below, explain what the data structure, "queue_t", is used for in the context of mutual exclusion. Provide detailed explanation of these code fragments. (The back of this page may be used if more space is needed for your answer)

```
typedef struct __lock_t {
    int flag;
    int guard;
    queue_t *q;
} lock_t;

void lock_init(lock_t *m) {
    m->flag = 0;
    m->guard = 0;
    queue_init(m->q);
}

void lock(lock_t *m) {
    while (TestAndSet(&m->guard, 1) == 1)
        ; //acquire guard lock by spinning
    if (m->flag == 0) {
        m->flag = 1; // lock is acquired
        m->guard = 0;
    } else {
        queue_add(m->q, getpid());
        m->guard = 0;
        park();
    }
}

void unlock(lock_t *m) {
    while (TestAndSet(&m->guard, 1) == 1)
        ; //acquire guard lock by spinning
    if (queue_empty(m->q))
        m->flag = 0; // let go of lock; no one wants it
    else
```

```

        unpark(queue_remove(m->q)); // hold lock (for next thread!)
m->guard = 0;
    }

```

จะมี queue ให้ process ที่มารอทำงาน โดย process ใน queue จะรออยู่
เมื่อ process ที่ทำงานอยู่ทำงานเสร็จ จะส่ง lock ให้ process ใน queue บังคับ
ปลุก process ที่ได้ lock มาทำงาน
แต่ถ้าไม่มี process ใน queue บังคับ ก็คือ unlock

ชื่อ รหัสประจำตัว ภาคเรียนที่

3: (15 points) Suppose we want to program a Web server; we could easily do so without using concurrency, as follows:

```

while(true)
{
    wait connection
    read from socket and parse url
    look up url contents in cache
    if (not in cache)
        fetch from disk
    put in cache
    send data to client
}

```

Because processing only one client at a time is not scalable, how can we adapt this code to work for a lot of clients? Outline the pseudo-code using producer-consumer concept. Explain how the concept is used in your pseudo-code.

เพิ่ม semaphore โดย init ให้
server = 1 → server ว่าใน client สามารถ
client = 0 → client ยังไม่พร้อมต่อ server
mutex = 1 → lock พร้อมในเอาไปใส่

จนบรรทัด wait connection เพิ่ม

sem_wait(&server); → เสิร์จว่า server ว่ายังมี

sem_wait(&mutex); → lock

ใต้บรรทัด send data to client เพิ่ม

sem_post(&mutex); → unlock

sem_post(&client); → ปลุก client มาทำงานต่อ

ชื่อรหัสประจำตัว.....ภาควิชา/ชั้นปี.....

ส่วนที่ 2

1. จากเนื้อหาที่เกี่ยวกับการจัดการอุปกรณ์เชื่อมต่อภายนอก (IO Device) จงตอบคำถามต่อไปนี้ (25 คะแนน)

- 1.1. อัลกอริทึมของ Disk Scheduling แบบ Shortest Position Time First (SPTF) มีหลักการทำงานอย่างไร ซึ่งอัลกอริทึมนี้ก่อให้เกิดปัญหา Starvation ได้ จงเสนอวิธีแก้ไขโดยละเอียด (10 คะแนน)

ดูจากอนั้นหัวอ่านอยู่ position ไหน แล้วไปอ่านข้อมูล
โดยจะไล่ลำดับจากระยะห่างว่า หัวที่จะอ่านกับปัจจุบัน จากน้อยไปมาก
ทำให้ข้อมูลที่อยู่ไกลจากตำแหน่งปัจจุบัน ถูกอ่านทีหลัง เลย starvation
งี้ → เปลี่ยน algorithm

- 1.2. อุปกรณ์ภายนอกแบบใดไม่เหมาะกับการใช้งานแบบ Interrupt พร้อมอธิบายเหตุผล (5 คะแนน)

device ที่ transfer data เร็ว
เพราะ เสียเวลา กับ การทำ interrupt

- 1.3. การส่งงานจาก Processor ไปยัง I/O Controller สามารถทำได้ 2 รูปแบบคือ แบบแรกใช้คำสั่ง Generic I/O ที่ทำงานร่วมกับ Memory-mapped I/O และแบบที่สองคือใช้คำสั่ง Special I/O ที่ส่งตรงไปยัง I/O Controller แต่ละตัว ทั้งสองแบบมีข้อดีข้อเสียแตกต่างกันอย่างไร พร้อมยกตัวอย่างสถานการณ์ที่ OS ควรใช้ทั้ง 2 วิธีส่งการ I/O Controller ตัวเดียว (10 คะแนน)

แบบ Memory-mapped I/O

↳ device register ถูก mapped ลง memory space

เพื่อใช้คำสั่ง store/load ปกติ

แบบ special I/O command

↳ ใช้คำสั่งพิเศษ read/write ไปที่ device register โดยตรง

สถานการณ์ที่ใช้ ทั้ง 2 → ไม่รู้

2. จากเนื้อหาที่เกี่ยวกับระบบไฟล์ (File System) จงตอบคำถามต่อไปนี้ (15 คะแนน)



2.1. จากแผนภาพแสดงความสัมพันธ์ของแต่ละโมดูลที่สำคัญ ที่จะทำให้แอปพลิเคชันต่างๆ (Application Programs) สามารถเข้าถึงข้อมูลที่อยู่ในฮาร์ดดิสก์ได้ (Harddisk Devices) จงอธิบายความรู้ที่มีต่อแผนภาพนี้ให้ได้มากที่สุด โดยเน้นการอธิบายในโมดูลที่เกี่ยวกับการทำงานในระบบปฏิบัติการ สำหรับโมดูลอื่นๆ อธิบายพอสังเขป (12 คะแนน)

device ถูกควบคุมโดย I/O control
 basic file system จะเป็นคนแปลรหัสข้อมูลของ
 แต่ละ device ให้เป็นรูปแบบเดียวกัน
 และจัดการเปลี่ยน physical ให้เป็น logical
 เพื่อให้ app เข้าไปใช้งาน

2.2. เทคโนโลยี Redundant Array of Inexpensive Disks มีความสามารถที่โดดเด่นอะไรบ้าง เมื่อเทียบกับการใช้ฮาร์ดดิสแบบธรรมดา (3 คะแนน)

เพิ่ม reliability เพิ่ม performance

3. จากความรู้ที่ได้จากการนำเสนอผลงานของ Assignment ที่ 2 จงตอบคำถามต่อไปนี้ (10 คะแนน)

3.1. จงวิจารณ์ความแตกต่างของการใช้ฟังก์ชัน malloc กับ vmalloc (4 คะแนน)

3.2. Completely Fair Scheduler เป็นตารางการจัดงานที่ลินุกซ์ Ubuntu ใช้อยู่ในเวอร์ชันปัจจุบัน ซึ่งสามารถแก้ปัญหา Starvation ได้อย่างไร (3 คะแนน)

3.3. คำว่า traps กับ interrupt เหมือนหรือแตกต่างกันอย่างไร (3 คะแนน)

ไม่ได้ทำ assign 2 คำ